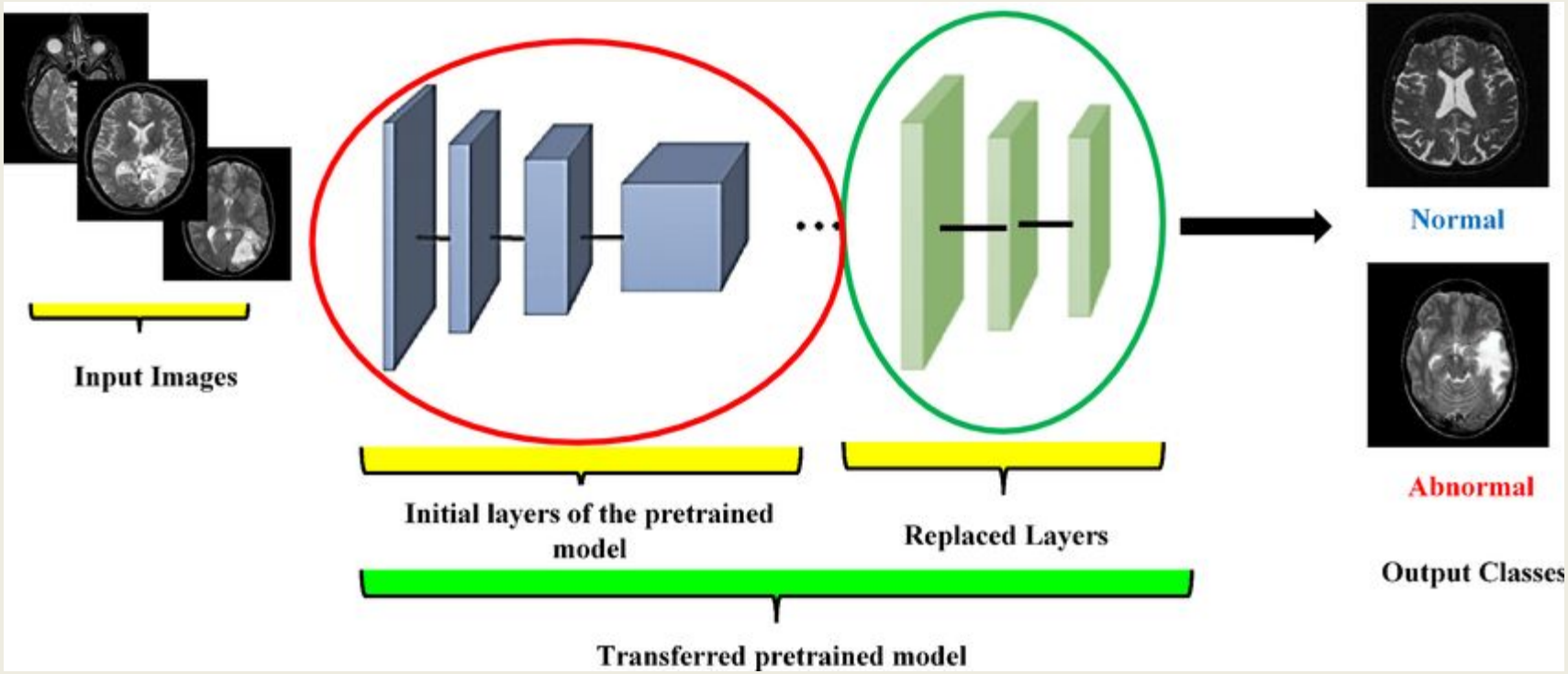
Pre-Trained Convolutional Networks

Pre-trained Models

- A pre-trained model - saved network that was previously trained on a large dataset, typically on a large-scale image-classification task.
- Either use the pre-trained model as it is or use transfer learning to customize this model to a given task.

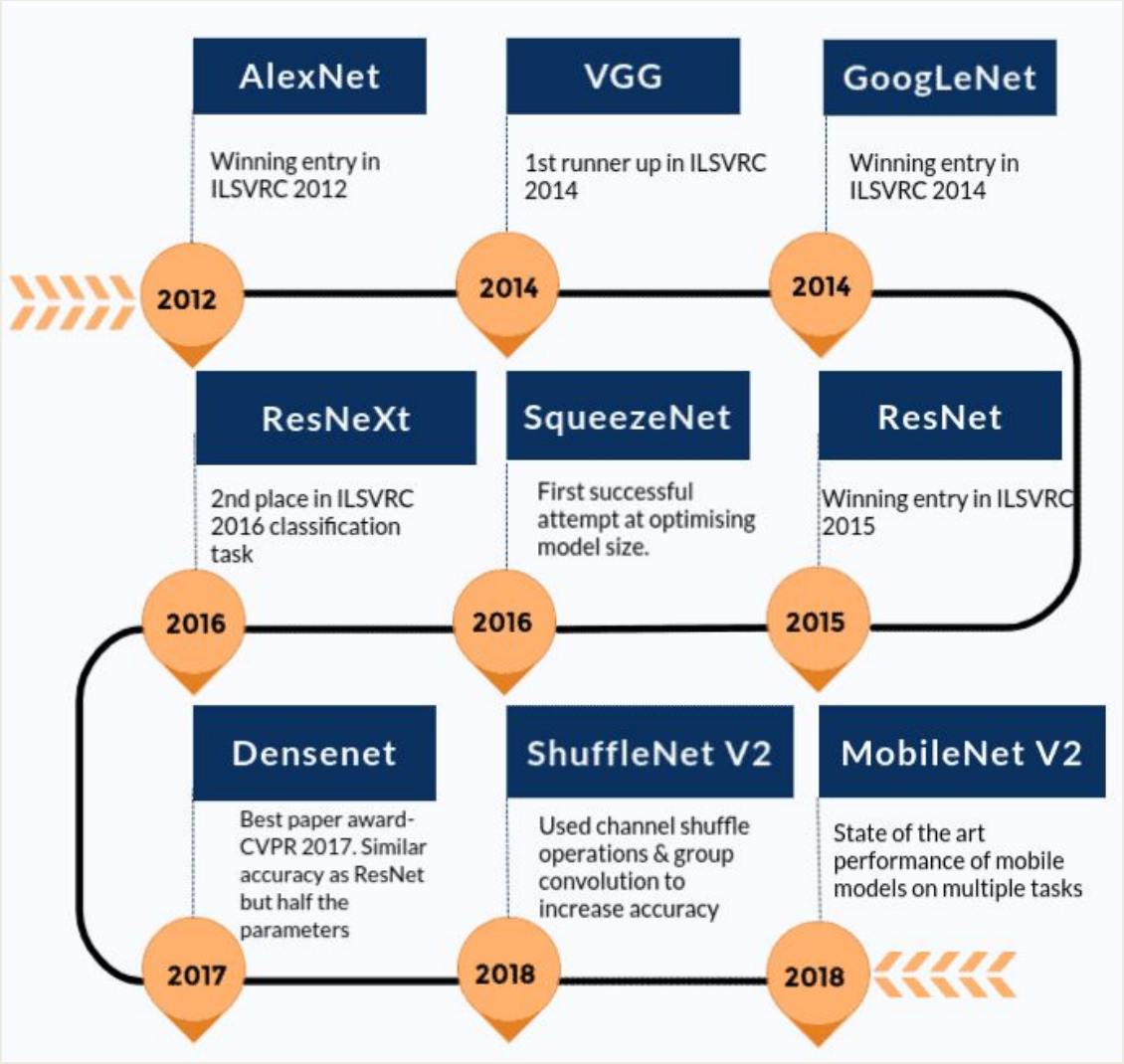
Pre-trained Models



Kaur, T., & Gandhi, T. K. (2020). Deep convolutional neural networks with transfer learning for automated brain image classification. *Machine Vision and Applications*, *31*(3), 1-16.

Pre-trained Models

- AlexNet
- VGG
- GoogLeNet
- ResNeXt
- SqueezeNet
- ResNet
- DenseNet
- ShuffleNetV2
- MobileNet V2



Why Pre-trained Models ?

- If a model is trained on a large and general enough dataset, this model will effectively serve as a generic model of the visual world.
- Can take advantage of these learned feature maps without having to start from scratch by training a large model on a large dataset.

Real Scenario

- if you want to build a self learning car.
- You can spend years to build a good image recognition algorithm from scratch or *you can take inception model* (*a pre-trained model*) from Google which was built on ImageNet data to identify images in those pictures.

Intuition

- Consider a problem on CrowdAnalytix platform – Identifying themes from mobile case images – an image classification problem
- Training Dataset - 4591 images
- Testing Dataset - 1200 images
- Target classes - 16

Intuition

- Simple MLP
- Accuracy – 6.8%

Layer (type)	Output Shape	Param #
flatten_4 (Flatten)	(None, 150528)	0
dense_9 (Dense)	(None, 500)	75264500
activation_9 (Activation)	(None, 500)	0
dropout_7 (Dropout)	(None, 500)	0
dense_10 (Dense)	(None, 500)	250500
activation_10 (Activation)	(None, 500)	0
dropout_8 (Dropout)	(None, 500)	0
dense_11 (Dense)	(None, 500)	250500
activation_11 (Activation)	(None, 500)	0
dropout_9 (Dropout)	(None, 500)	0
dense_12 (Dense)	(None, 16)	8016
activation_12 (Activation)	(None, 16)	0

Total params: 75,773,516
Trainable params: 75,773,516
Non-trainable params: 0

Epoch 10/10

50/50 [==============================] – 21s – loss: 15.0100 – acc: 0.0688

Intuition

- CNN
- 32 filters of 5*5
- Activation function – ReLU
- Max pooling layer of size 4*4
- Dropout – 0.5
- Accuracy – 15%

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 220, 220, 32)	2432
activation_27 (Activation)	(None, 220, 220, 32)	0
max_pooling2d_7 (MaxPooling2	(None, 55, 55, 32)	0
conv2d_8 (Conv2D)	(None, 51, 51, 32)	25632
activation_28 (Activation)	(None, 51, 51, 32)	0
max_pooling2d_8 (MaxPooling2	(None, 12, 12, 32)	0
conv2d_9 (Conv2D)	(None, 8, 8, 64)	51264
activation_29 (Activation)	(None, 8, 8, 64)	0
max_pooling2d_9 (MaxPooling2	(None, 2, 2, 64)	0
flatten_8 (Flatten)	(None, 256)	0
dense_21 (Dense)	(None, 64)	16448
activation_30 (Activation)	(None, 64)	0
dropout_12 (Dropout)	(None, 64)	0
dense_22 (Dense)	(None, 16)	1040
activation_31 (Activation)	(None, 16)	0

Total params: 96,816
Trainable params: 96,816
Non-trainable params: 0

Epoch 10/10

50/50 [==============================] – 21s – loss: 13.5733 – acc: 0.1575

Intuition

- VCG16 – pretrained model on Imagenet

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
flatten (Flatten)	(None, 25088)	0
fc1 (Dense)	(None, 4096)	102764544
fc2 (Dense)	(None, 4096)	16781312
predictions (Dense)	(None, 1000)	4097000
dense_1 (Dense)	(None, 16)	16016

Total params: 138,373,560
Trainable params: 138,373,560
Non-trainable params: 0

Epoch 10/10

50/50 [==============================] - 21s - loss: 02.372 - acc: 0.8273

How to use Pre-trained Models

- Many pre-trained models are available in the keras library
- **Imagenet** data set - widely used to build various architectures – has 1.2M images to create a generalized model.

How to use Pre-trained Models

- **Feature extraction** –
  - Use a pre-trained model as a feature extraction mechanism.
  - Can remove the output layer(the one which gives the probabilities for being in each of the 1000 classes) and then use the entire network as a fixed feature extractor for the new data set.
- **Use the Architecture of the pre-trained model**
  - Use the architecture of the model while we initialize all the weights randomly and train the model according to our dataset again.
- **Train some layers while freeze others** –
  - Use a pre-trained model and train them partially.
  - Keep the weights of initial layers of the model frozen while we retrain only the higher layers.
  - Can try and test as to how many layers to be frozen and how many to be trained.

How to use Pre-trained Models

- **Scenario 1 - Size of the Data set is small while the Data similarity is very high**
  - Do not need to retrain the model.
  - Customize and modify the output layers according to our problem statement.
  - Use the pretrained model as a feature extractor.
  - Example - Classify a cats or dogs.
  - Images we need to identify would be similar to imagenet, however we just need two categories as my output – cats or dogs.
  - In this case all we do is just modify the dense layers and the final softmax layer to output 2 categories instead of a 1000.

How to use Pre-trained Models

- **Scenario 2 – Size of the data is small as well as data similarity is very low**
  - Can freeze the initial (let's say k) layers of the pre-trained model and train just the remaining(n-k) layers again.
  - Top layers would then be customized to the new data set.
  - Since the new data set has low similarity it is significant to retrain and customize the higher layers according to the new dataset.
  - The small size of the data set is compensated by the fact that the initial layers are kept pretrained (which have been trained on a large dataset previously) and the weights for those layers are frozen.

How to use Pre-trained Models

- **Scenario 3 – Size of the data set is large however the Data similarity is very low**
  - We have a large dataset, our neural network training would be effective.
  - Since the data we have is very different as compared to the data used for training our pretrained models.
  - The predictions made using pretrained models would not be effective.
  - Hence, its best to train the neural network from scratch according to your data.

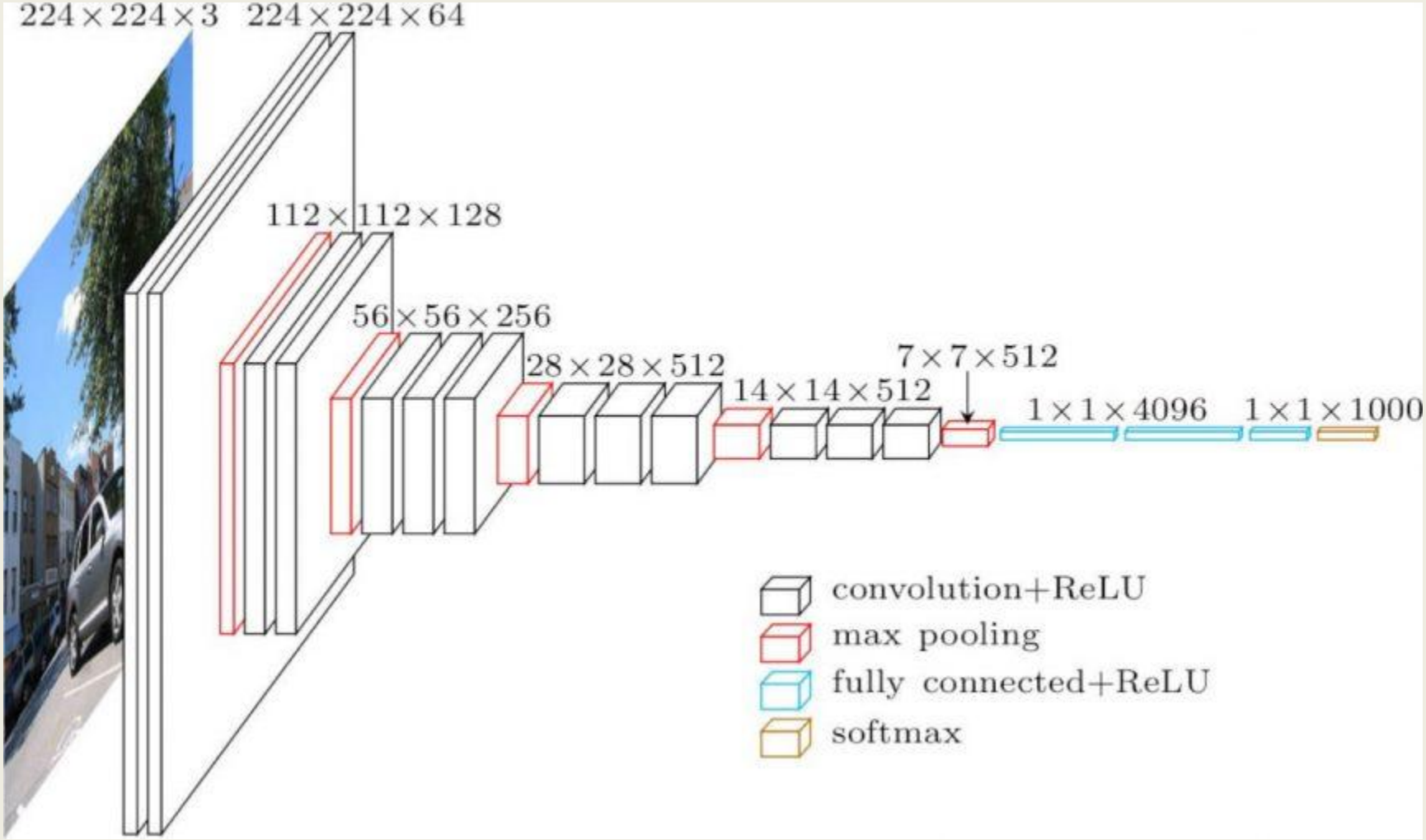
How to use Pre-trained Models

- **Scenario 4 - Size of the data is large as well as there is high data similarity**
  - Ideal situation.
  - In this case the pretrained model should be most effective.
  - The best way to use the model is to retain the architecture of the model and the initial weights of the model.
  - Then we can retrain this model using the weights as initialized in the pre-trained model.

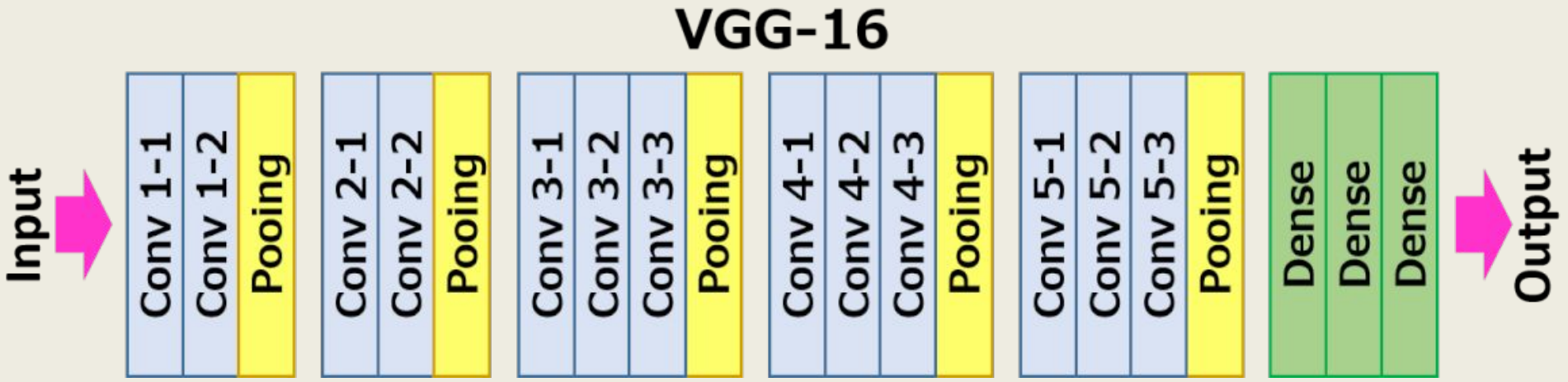
VGG - 16

- Very Deep Convolutional Networks for Large-Scale Image Recognition(VGG-16)
  - The VGG-16 - most popular pre-trained models for image classification.
  - Introduced in the famous ILSVRC 2014 Conference, it was and remains THE model to beat even today.
  - Developed at the Visual Graphics Group at the University of Oxford, VGG-16 beat the then standard of AlexNet and was quickly adopted by researchers and the industry for their image Classification Tasks.

VGG – 16 Architecture



VGG – 16 Architecture

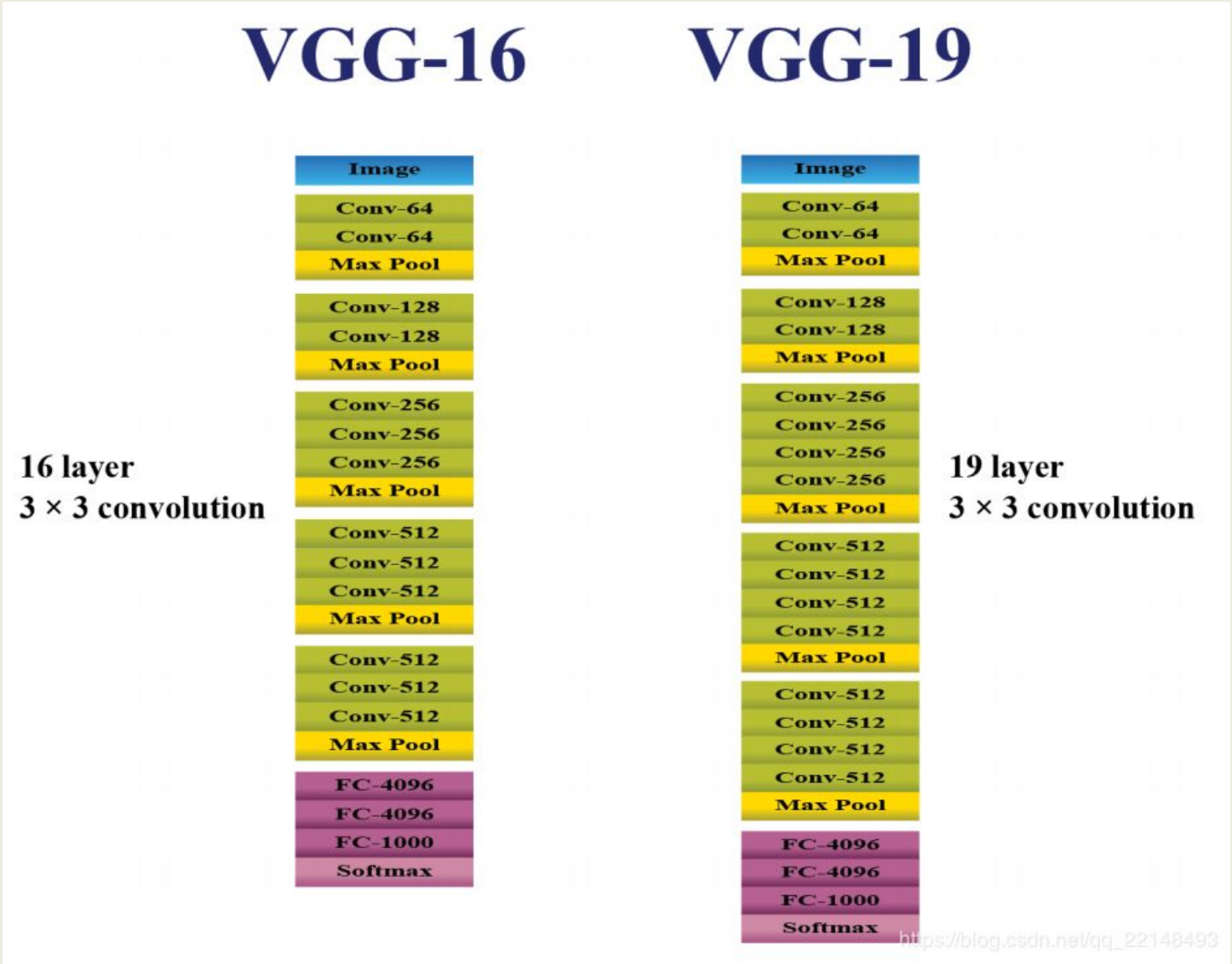


Convolution layers – 13

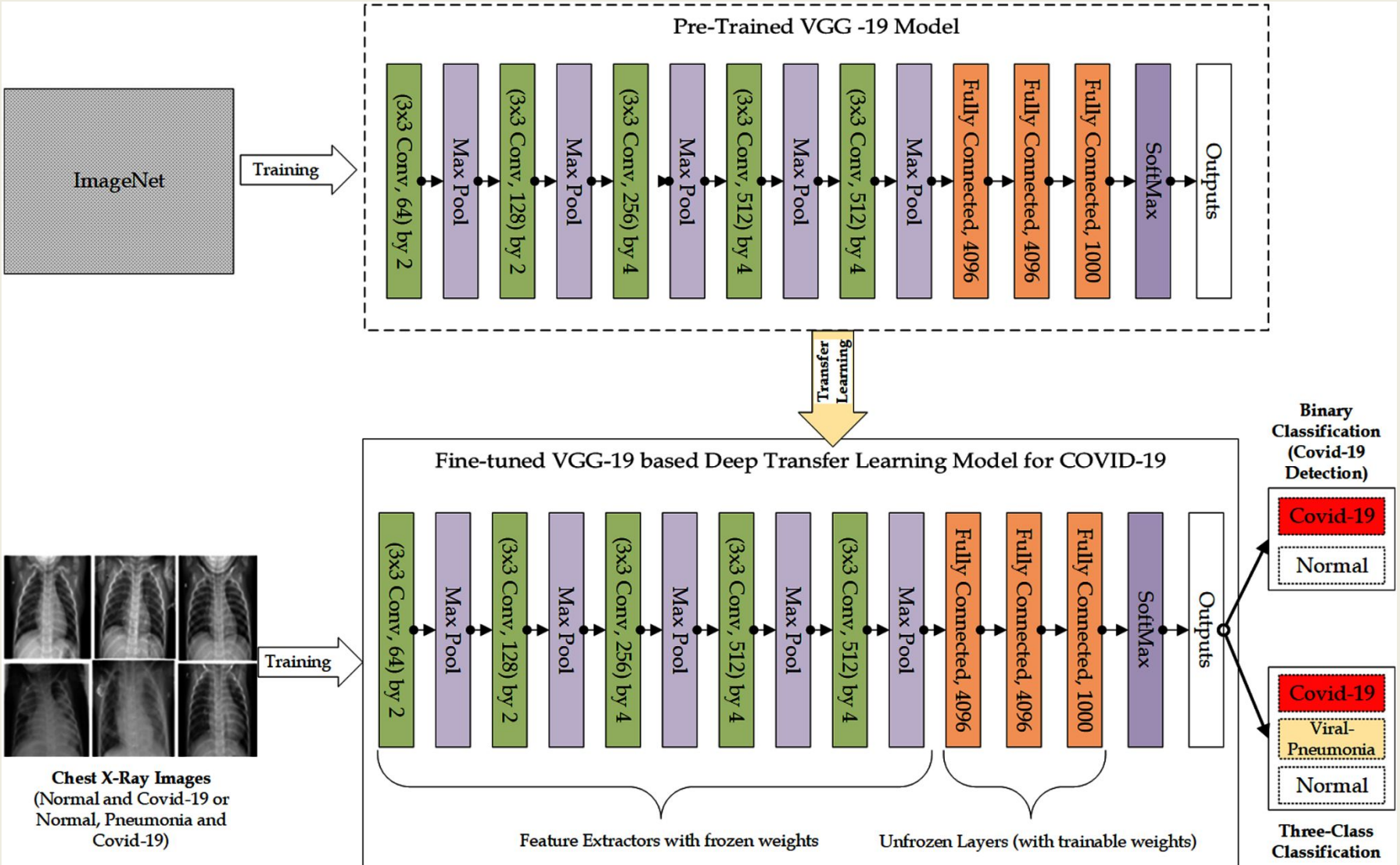
Pooling layers – 5

Dense layers – 3

VGG-19 Architecture



Fine-tuned VGG-19 Transfer Learning Model



LeNet and AlexNet

Inception Model

- Inception model (GoogLeNet) is a convolutional neural network which helps in classifying the different types of objects on images
- Uses ImageNet dataset for training process.
- In the case of Inception, images need to be 299x299x3 pixels size.
- Inception Layer is a combination of 1×1, 3×3 and 5×5 convolutional layer with their output filter banks concatenated into a single output vector forming the input of the next stage.

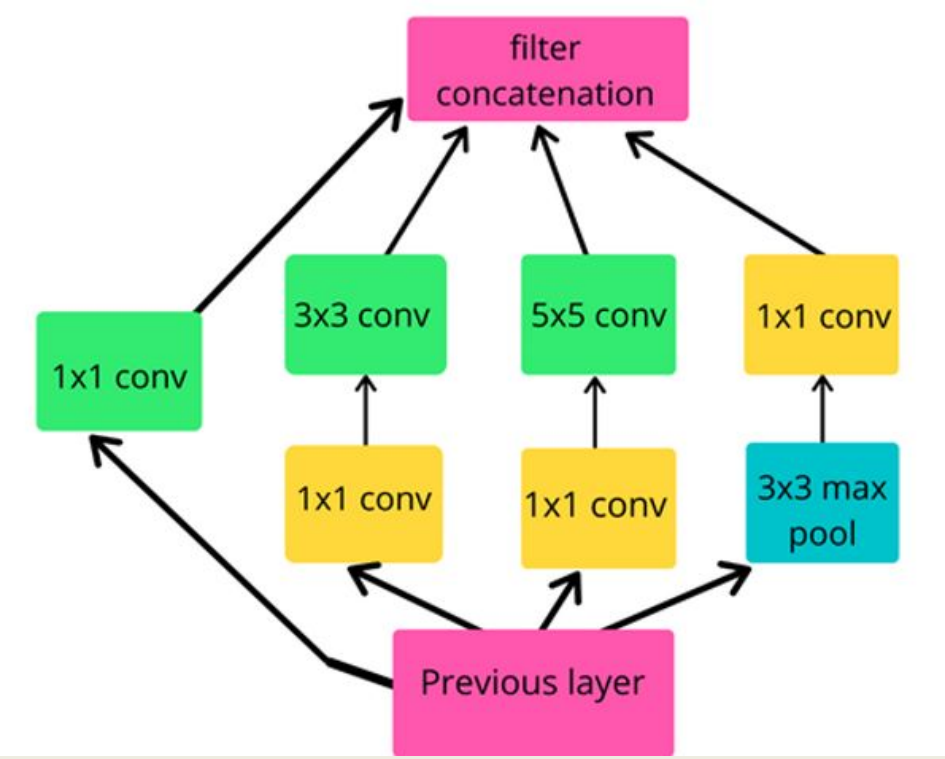
Inception Model

Types of Inception

- Inception v1
- Inception v2
- Inception v3

Inception v1 (Naïve version)

- Naïve version performs convolution on an input, with 3 different sizes of filters i.e. 1x1, 3x3 and 5x5 convolution.
- Max pooling is also performed.
- The output's layers are then concatenated and passed to the next Inception module.



Inception v1 (Naïve version)

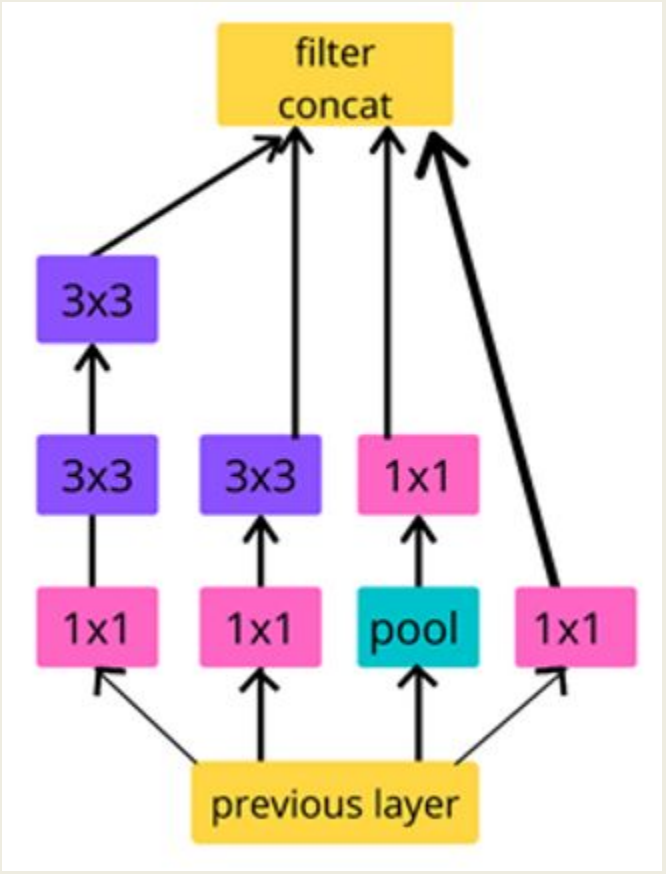
- GoogLeNet
  - 9 such inception modules stacked linearly.
  - It is 22 layers deep or 27, including the pooling layers.
  - Uses global average pooling at the end of the last inception module.

Inception v2

- Increases the number of filters which leads to increases the accuracy and reduces the computational complexity.
- Factorization method is used to reduce the computational complexity.
- Inceptionv2 is wider than deeper.

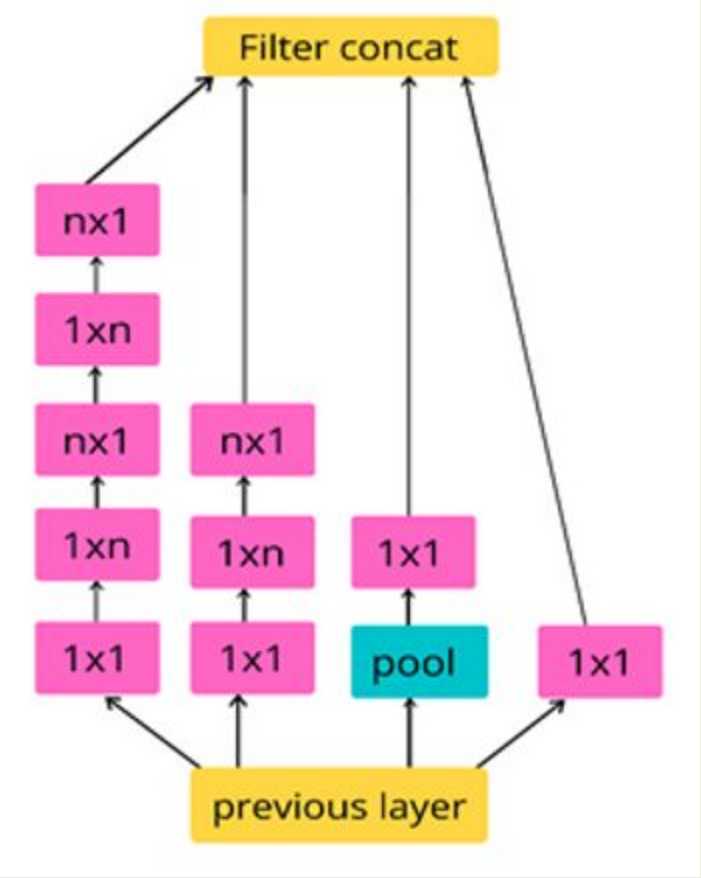
Inception v2

- In Inception v2 architecture, 5×5 convolution is replaced by the two 3×3 convolutions.
- Decreases computational time and thus increase computational speed because a 5×5 convolution is 2.78 more expensive than 3×3 convolution.
- So, using two 3×3 layers instead of 5×5 boost the performance of architecture.



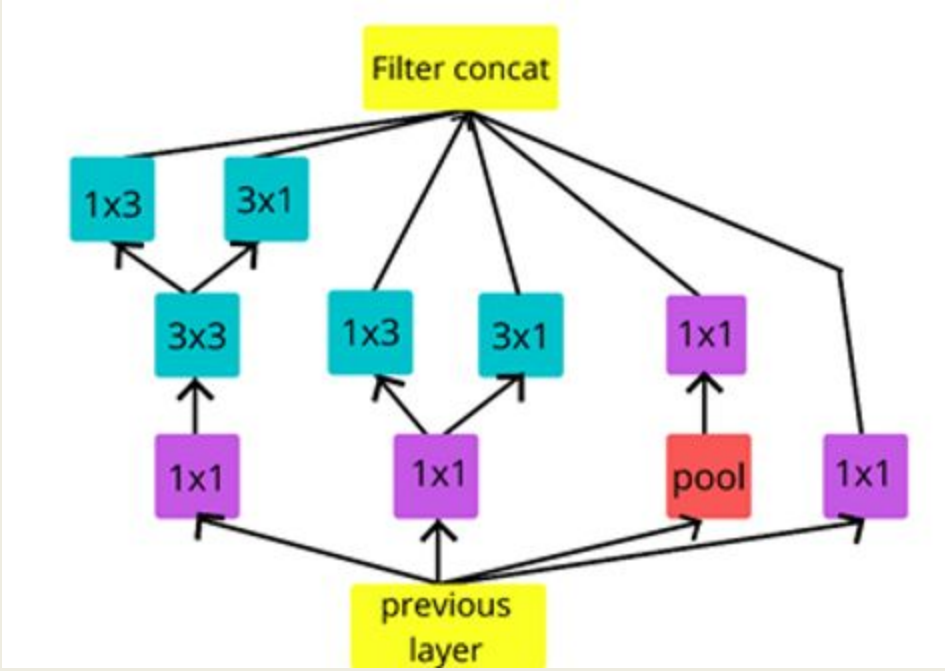
Inception v2

- This architecture also converts nxn factorization into 1xn and nx1 factorization.
- 3×3 convolution can be converted into 1×3 then followed by 3×1 convolution which is 33% cheaper in terms of computational complexity as compared to single 3×3 convolution.



Inception v2

- To deal with the problem of the representational bottleneck, the feature banks of the module were expanded instead of making it deeper.
- This would prevent the loss of information that causes when we make it deeper.

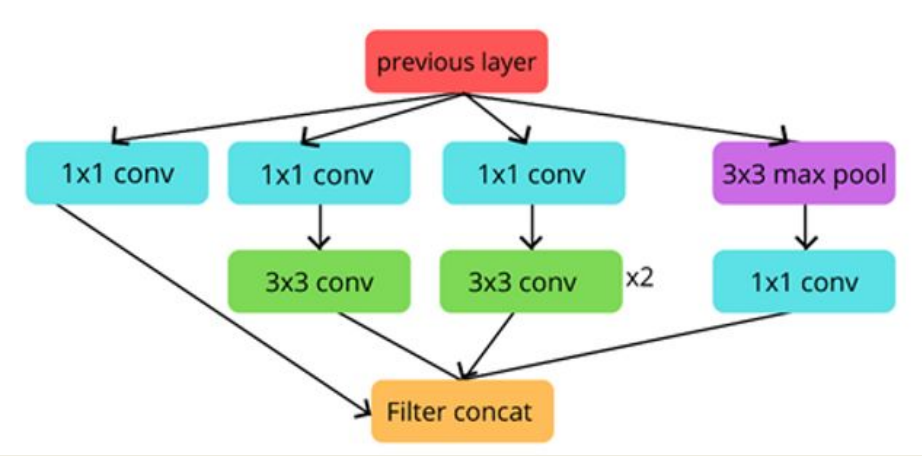


Inception v3

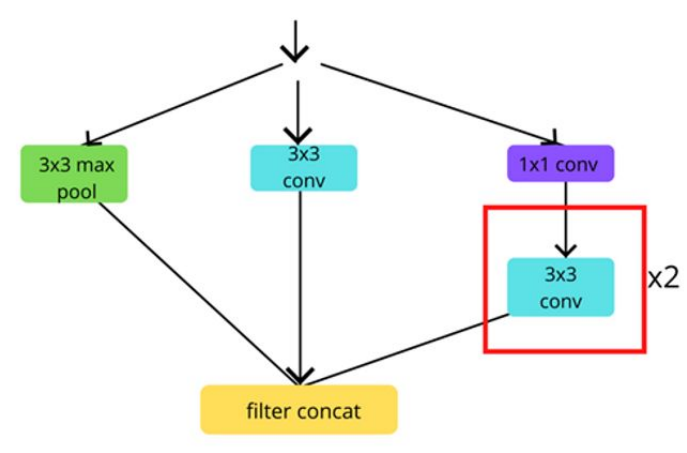
- version 3 adds some of the following modifications
  - Factorized 7x7 convolutions.
  - Batch Normalization in the Auxiliary Classifiers (to overcome the problem of vanishing gradient).
  - Use of RMSprop optimizer.
  - Asymmetric convolutions.
  - Smaller convolutions.
  - Grid size reduction.
  - Label Smoothing (A type of regularizing component added to the loss formula that prevents the network from becoming too confident about a class. Prevents overfitting).
- V3 has network of 48 layers
- Trained on some part of ImageNet dataset and in 1000 different classes.

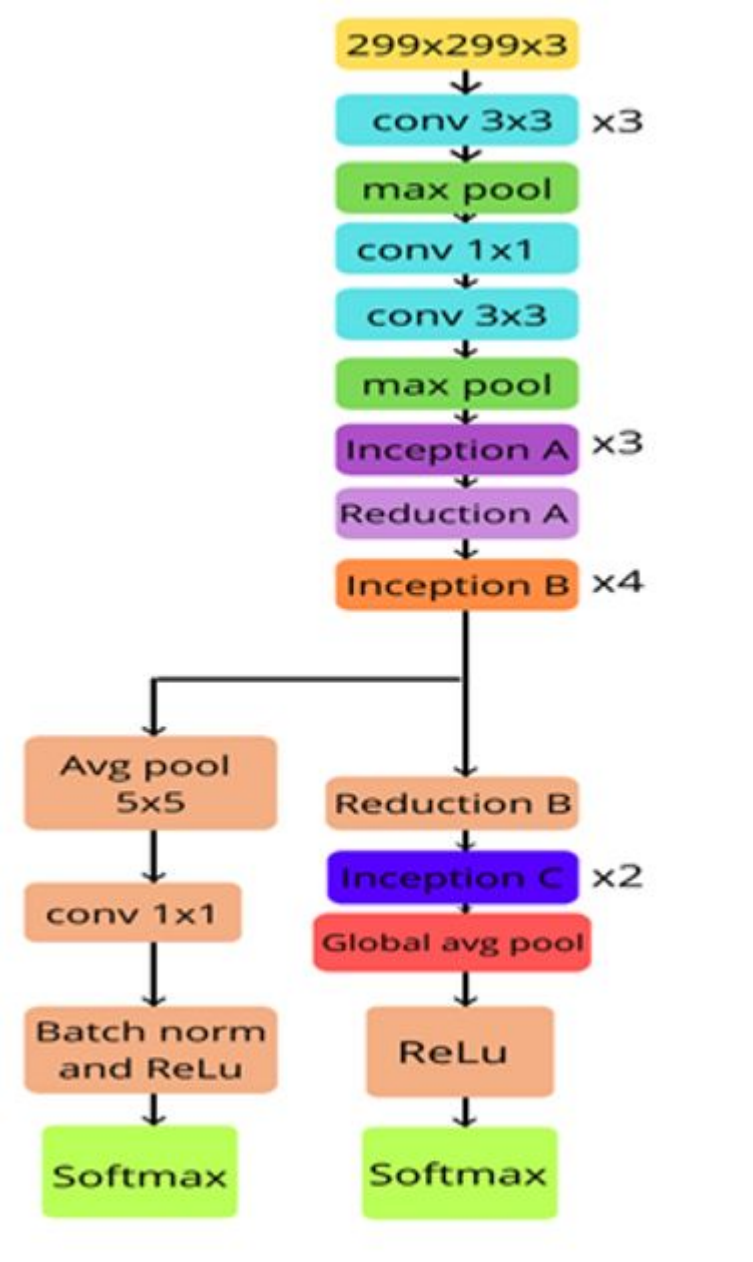
Inception v3

Inception A



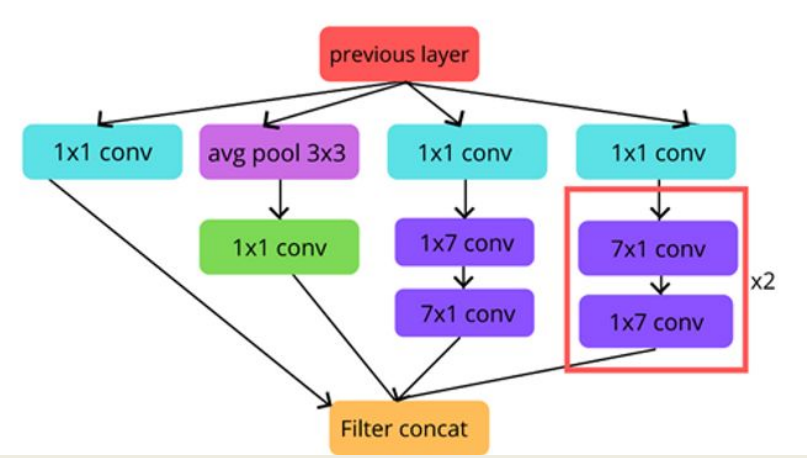
Reduction A



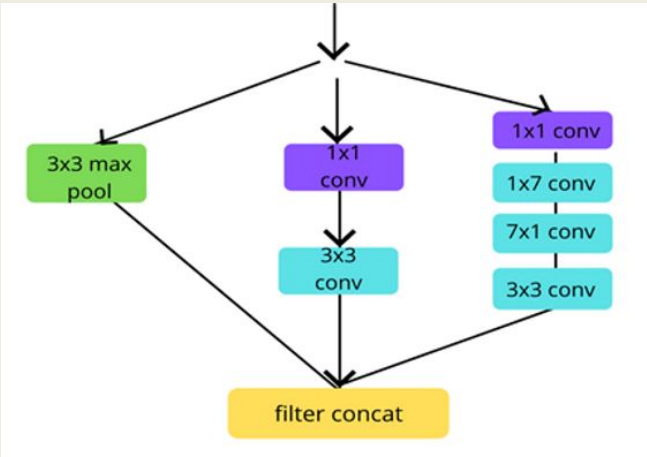


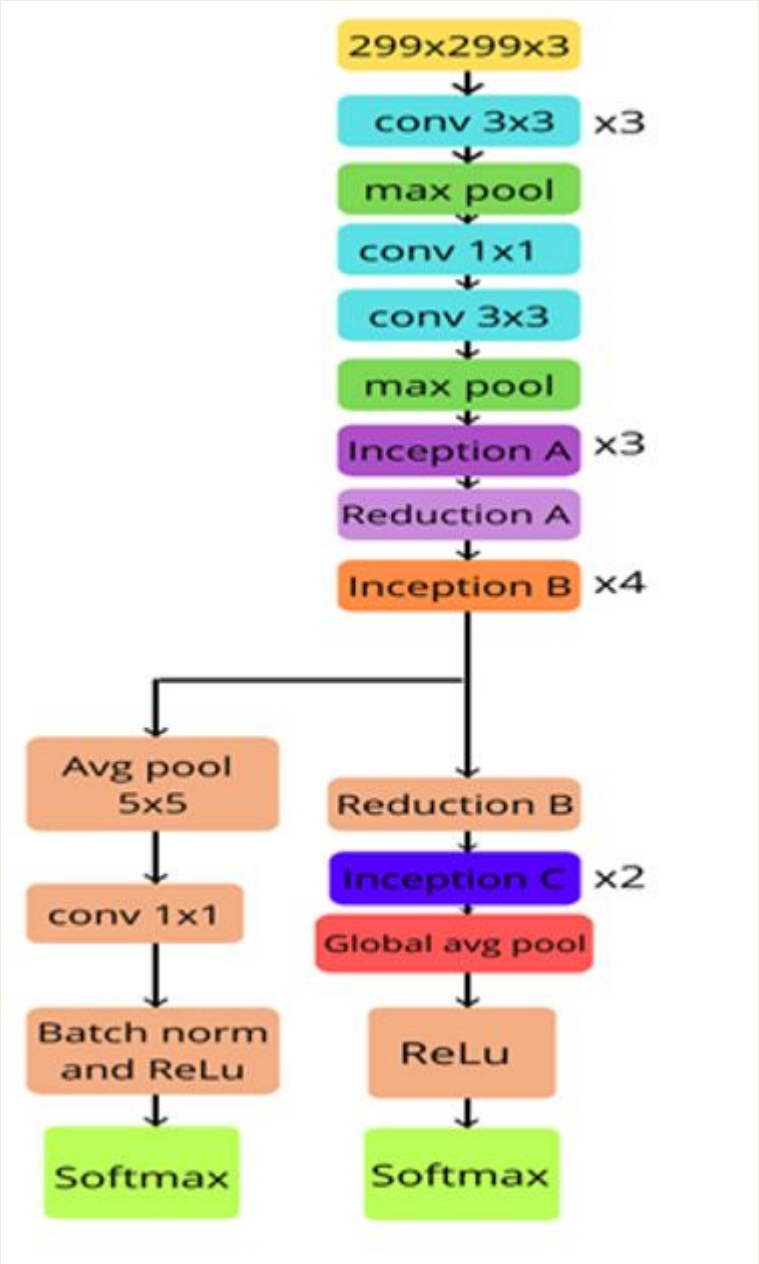
Inception v3

Inception B



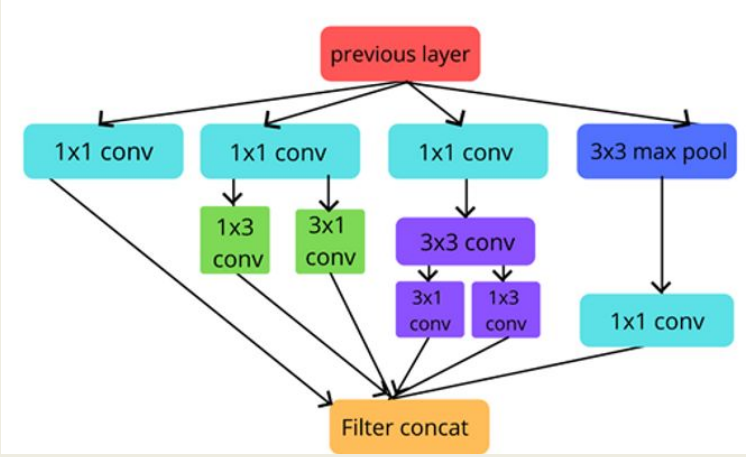
Reduction B

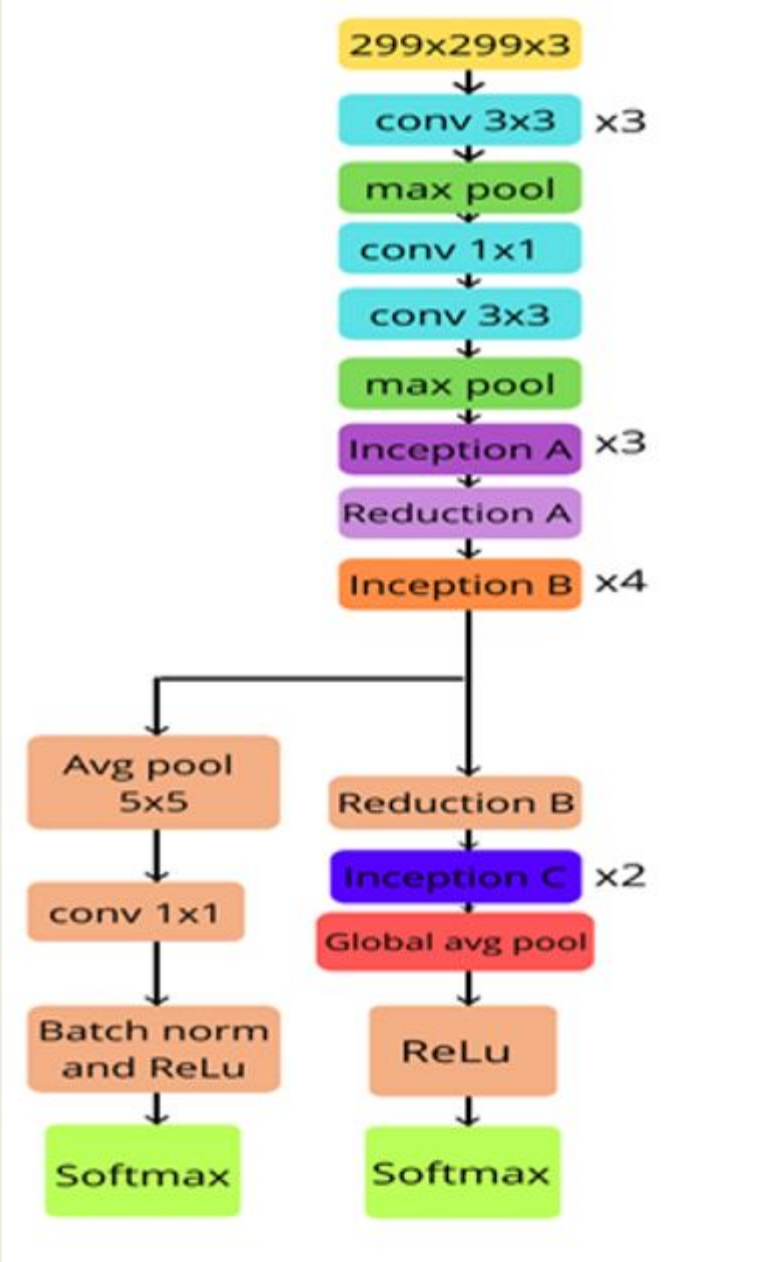




Inception v3

Inception B





Thank You